Conexión Entre Clientes Mediante Hilos

Nicolle Bustamante Hernández
Ingeniería de Sistemas y Computación
Universidad Tecnológica de Pereira
Pereira, Colombia
n.bustamante@utp.edu.co

***Resumen*—En esta práctica se desarrolló un sistema de chat en Python sobre sockets TCP, con un servidor que actúa como intermediario entre varios clientes y usa hilos para atender cada conexión de forma concurrente. Sobre esa base se implementaron mensajería pública y privada, validación de nombres de usuario, transferencia de archivos con confirmación del destinatario y un historial local de archivos aceptados y rechazados. Las pruebas se realizaron sobre dos máquinas virtuales conectadas por una red privada, y permitieron verificar el paso de mensajes entre clientes y el comportamiento de los mecanismos de control implementados.**

***Index Terms*—Sockets TCP, comunicación entre procesos, cliente-servidor, Python, concurrencia, transferencia de archivos, RFC 9293.**

I. Objetivo

El objetivo de esta práctica fue construir un sistema de chat en el que varios clientes se comuniquen entre sí a través de un servidor intermedio, con hilos dedicados a cada conexión para que ninguno bloquee a los demás. El servidor no debe hablar directamente con los clientes: recibe los mensajes de un cliente y los redirige al destinatario o destinatarios correspondientes. El resultado esperado es un paso de mensajes consistente entre las máquinas cliente, con reglas explícitas sobre quién puede ver cada mensaje y cómo se confirman las transferencias de archivos.

II. Primera aproximación

Para empezar se tomó el código base proporcionado en la guía, que consta de dos archivos: un servidor que atiende a varios clientes con hilos y los reenvía, y un cliente que abre dos hilos internos (uno para enviar, uno para recibir). El servidor mantiene una lista de clientes conectados y, cada vez que alguien envía un mensaje, lo difunde a todos los demás.

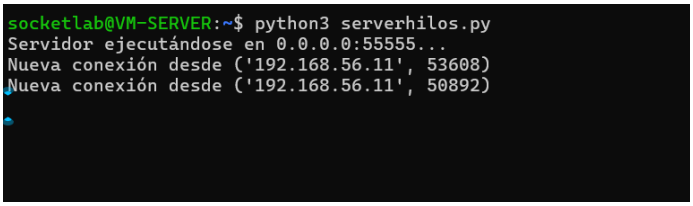


Figura 1. Servidor del código base ejecutándose en la VM-SERVIDOR, con la primera conexión aceptada desde la VM-CLIENTE.

Se ejecutó el servidor en la VM-SERVIDOR y dos instancias del cliente en la VM-CLIENTE, cada una en su propia terminal SSH. Al escribir en una de ellas, el mensaje apareció casi al instante en la otra, lo cual confirmó que el paso de mensajes funcionaba. Sin embargo, al probarlo con más de dos clientes se notó algo raro: si dos personas elegían el mismo nickname, el servidor no lo detectaba, y los mensajes podían quedar ambiguos o dirigidos a la persona equivocada. No había ninguna validación, así que dos clientes podían llamarse igual sin que nadie lo impidiera.

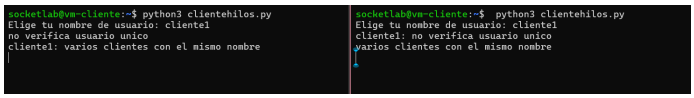


Figura 2. Evidencia del problema: dos clientes conectados con el mismo nickname, sin que el servidor lo detecte.

El problema no era grave, pero dejaba el sistema en un estado inconsistente: si dos clientes se llamaban ‘cliente1‘, cuando alguien enviara un mensaje privado a ‘cliente1‘, no había forma de saber a cuál de las dos le llegaría. Se decidió corregirlo antes de seguir agregando funcionalidades.

III. Corrección del problema de nicknames repetidos

La corrección consistió en dos cambios en el servidor. Primero, la lista de clientes pasó de ser una lista de sockets a un diccionario que asocia cada socket con su nickname (`clientes = {socket: nickname}`). Segundo, cuando un cliente se conecta, antes de aceptarlo se verifica que su nickname no esté ya en uso. Si lo está, el servidor responde con un error y le pide al cliente que elija otro.

También se agregaron validaciones básicas: el nickname no puede estar vacío, no puede tener espacios, no puede empezar con ‘@‘ ni contener ‘:‘, y no puede llamarse ‘todos‘ (porque esa palabra se reserva para los mensajes de difusión). Si alguna falla, el servidor responde con un mensaje de error y el cliente vuelve a preguntar.

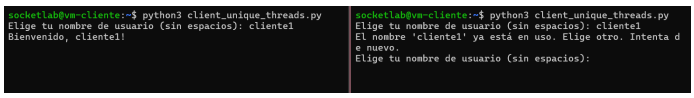


Figura 3. El servidor rechaza un nickname repetido y le pide al cliente que elija otro.

Después de la corrección, dos clientes ya no podían tener el mismo nombre. El cliente, al recibir un error, vuelve a pedir el nickname sin necesidad de reconectarse, lo cual se logró con un bucle de handshake que se repite hasta obtener un 'OK' del servidor.

IV. Primera versión: chat con elección de destinatario

Con el problema de los nicknames resuelto, la primera versión completa del sistema agregó la posibilidad de elegir destinatario. Hasta ese momento todos los mensajes iban a todos; ahora se podían usar dos comandos:

- `@todos: mensaje` para enviar a todos los clientes conectados excepto al emisor.
- `@nickname: mensaje` para enviar un mensaje privado a un cliente específico.
- `mensaje` (sin prefijo) para enviar a todos por defecto.

En el servidor, la función `procesar_mensaje` interpreta el texto y decide si es un mensaje a todos o un privado. En el cliente, los mensajes se envían siempre terminados en `\n`, y el hilo de recepción lee línea por línea usando un buffer, para no depender del tamaño exacto del `recv()`.

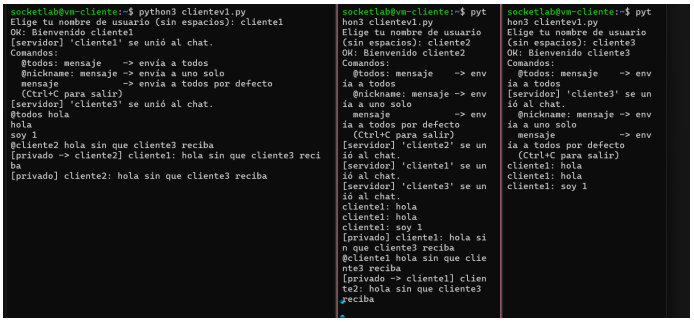


Figura 4. Chat público y privado entre clientes. Se muestra el envío de mensajes a todos los clientes conectados y la comunicación privada entre dos clientes, con la confirmación de recepción al emisor.

También se agregó el comando `@lista`, que devuelve la lista de usuarios conectados en ese momento. Esto resultó útil para saber a quién se le podía escribir, sobre todo cuando varios clientes entraban y salían del chat.

V. Segunda versión: envío de archivos con base64

La siguiente extensión fue el envío de archivos. El reto técnico es que los sockets transmiten bytes, y TCP no conserva las fronteras entre mensajes, así que no se puede enviar un archivo binario suelto y esperar que llegue en un solo `recv()`. Para resolverlo se usó una codificación en base64: el archivo se lee en modo binario, se codifica a una cadena de caracteres ASCII, y se envía como parte de una línea delimitada por `\n`. En el receptor, la cadena se decodifica y el archivo se guarda en disco.

El formato del mensaje es:

```
@archivo:destinatario:nombre_archivo:
    contenido_en_base64
```

El base64 tiene la ventaja de que no contiene `\n`, así que no rompe el delimitador de la línea. Además, funciona para cualquier tipo de archivo (texto, PDF, imágenes), no solo para texto plano.

En el cliente, el comando correspondiente es `@archivo:nickname:ruta` o `@archivo:todos:ruta`. El cliente lee el archivo, lo codifica en base64 y lo envía al servidor. El servidor identifica el prefijo `@archivo:`, valida el tamaño (con un máximo de 5 MB) y reenvía el contenido al destinatario o a todos.

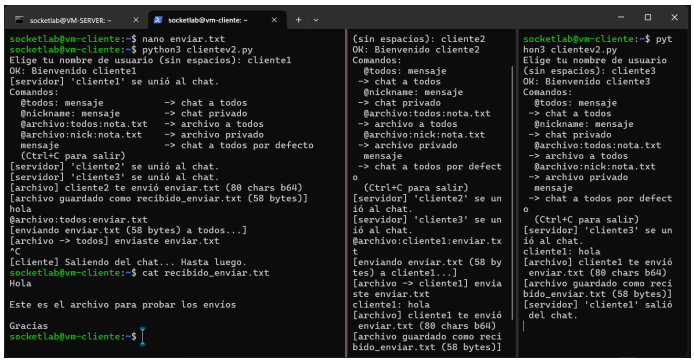


Figura 5. Envío y recepción de un archivo de texto entre clientes. Se muestra la notificación de recepción, el almacenamiento del archivo en disco con el prefijo `recibido_` y la verificación de su integridad mediante `md5sum`.

Esta versión ya permitía enviar cualquier tipo de archivo hasta 5 MB. Sin embargo, el archivo se guardaba automáticamente, sin preguntarle al receptor si quería aceptarlo. Ese comportamiento no era ideal, porque cualquiera podía llenar el disco del receptor con archivos no deseados.

VI. Tercera versión: confirmación del receptor

La última mejora consistió en agregar un paso de confirmación antes de transferir el archivo. Ahora el servidor no reenvía el contenido directamente: primero envía una oferta con un identificador único, y el receptor decide si acepta o rechaza. Solo si acepta, el servidor le envía el contenido. Además, se implementó un historial local de archivos, que registra tanto los archivos aceptados como los rechazados, con la fecha, el emisor, el nombre del archivo, el tamaño y la ruta donde se guardó. Este historial se mantiene en memoria durante la sesión y además se escribe en un archivo `historial_archivos.txt` para que persista entre ejecuciones.

El flujo completo es el siguiente:

1. El emisor envía `@archivo:destino:ruta`. El cliente codifica el archivo en base64 y lo envía al servidor.
2. El servidor guarda el archivo en una tabla de ofertas pendientes, con un identificador único, y envía al receptor una notificación del tipo `@archivo_oferta:id:emisor:nombre:tamaño`.
3. El receptor ve la oferta, con el nombre del emisor, el nombre del archivo y su tamaño estimado.

4. El receptor responde con `@aceptar <id>` o `@rechazar <id>`.
5. Si acepta, el servidor le envía el contenido en base64 y él lo guarda en disco. Si rechaza, no se transfiere nada.
6. En ambos casos, el historial local del receptor se actualiza con el registro correspondiente, y el emisor original recibe una notificación de si su archivo fue aceptado o rechazado.

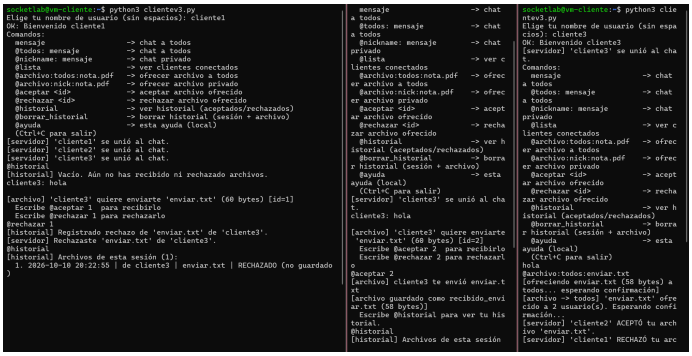


Figura 6. Flujo completo del sistema: dos clientes conectados, intercambio de mensajes, oferta de archivo, confirmación y consulta del historial local.

Este flujo también funciona cuando el archivo se ofrece a todos los clientes conectados: cada uno recibe su propia oferta con su propio identificador, y cada uno decide de forma independiente. El servidor lleva el control de todas las ofertas pendientes y las elimina cuando se responden o cuando el emisor o el receptor se desconectan.

VII. Conclusiones

En un sistema de mensajería, la complejidad real no está en enviar bytes sino en decidir qué garantiza cada mensaje. Sin un protocolo explícito no se puede distinguir un mensaje de un archivo, ni tratar de forma separada a dos clientes con el mismo nombre. El diseño del protocolo terminó siendo más importante que la concurrencia misma, porque los hilos resuelven que varios clientes puedan hablar a la vez, pero no resuelven qué significa cada cosa que se envían.

La confirmación de archivos mostró además que aceptar un archivo no es solo guardarlo: implica saber quién lo envió, con qué identificador, y qué hacer si un extremo se desconecta a mitad de la transferencia. Y el historial dejó claro que el estado de una aplicación distribuida no vive solo en el servidor, sino que cada cliente necesita llevar su propio registro de lo que aceptó, rechazó o dejó pasar.

Referencias

[1] IBM Corporation, *Cómo funcionan los sockets*, IBM Documentation. [En línea]. Disponible en: https://www.ibm.com/docs/es/i/7.5.0?topic=programming-how-sockets-work

[2] F. Gutiérrez Isaza, *Laboratorio N.º 1: Comunicación entre procesos mediante sockets TCP*, Universidad Tecnológica de Pereira, periodo 2026-2.